Assignment 04 Estimation Functions

Joy Polk

Purpose

The purpose of this assignment is to develop reuseable functions for estimating multiple linear regression models using matrix operations in a Click-to-Run file that can be rendered as HTML and PDF across various operating systems (e.g., Mac, Windows, Linux).

Core: Task 1

In your project repository, write (at minimum) two functions built on the matrix-based approach: a fitting function that takes an outcome and predictors (a matrix or a data structure you document — your design choice) and returns the coefficients, their standard errors, the residual variance, and the fitted values; and a prediction function that takes a fitted result and new predictor values and returns predicted outcomes.

Two reusable functions were developed and saved in `R/regression_functions.R`. The first, `fit_matrix_reg()`, estimates multiple linear regression models using matrix calculations. The second, `predict_matrix_reg()`, generates predicted outcomes from the estimated coefficients and new predictor values.

Core: Task 2

Document both functions with comments stating their inputs, outputs, and assumptions. Someone opening the file cold (for instance, you in November) should know what goes in, what comes out, and what the function assumes about its inputs.

Both functions include comments describing their inputs, outputs, and assumptions. The fitting function accepts a numeric outcome vector and a predictor matrix without an intercept column, which is added internally. The prediction function accepts the fitted model and new predictor values in the same order used during estimation. Both functions require complete numeric data, and the fitting function additionally requires a full-rank design matrix with more observations than estimated coefficients.

```r
#load functions
source("R/regression_functions.R")

#Load dataset
motivation_data <- read.csv("data/motivation.csv")

#Inspect dataset
head(motivation_data)
```

```
  sibling parent peer motivation
1    3.88   8.05 3.49       9.27
2    6.13   9.44 2.94      10.02
3    4.49   6.54 5.00      10.10
4    5.42   8.67 2.86       9.03
5    6.16   9.57 3.26      11.97
6    4.97   8.53 3.05      10.21
```

Core: Task 3

Demonstrate the fitting function on motivation.csv with all three predictors (parent, sibling, peer) predicting motivation, and show explicitly in the rendered document that your results match lm() in R or statsmodels in Python to numerical tolerance; a printed comparison or an allclose()-style check, visible in the output.

```r
#Prepare outcome and predictors
y <- motivation_data$motivation

X <- as.matrix(
  motivation_data[, c("parent", "sibling", "peer")]
)

#Fit matrix-based regression
matrix_model <- fit_matrix_reg(y, X)

#Fit regression using lm()
lm_model <- lm(
  motivation ~ parent + sibling + peer,
  data = motivation_data
)

#Compare regression coefficients
```

```r
coefficient_comparison <- data.frame(
  Matrix = as.vector(matrix_model$coefficients),
  LM = as.vector(coef(lm_model))
)

rownames(coefficient_comparison) <- names(coef(lm_model))

coefficient_comparison
```

```
               Matrix        LM
(Intercept) 3.4182221 3.4182221
parent      0.4708032 0.4708032
sibling     0.2301395 0.2301395
peer        0.5236967 0.5236967
```

```r
#Compare standard errors
se_comparison <- data.frame(
  Matrix = as.vector(matrix_model$standard_errors),
  LM = as.vector(summary(lm_model)$coefficients[, "Std. Error"])
)

rownames(se_comparison) <- names(coef(lm_model))

se_comparison
```

```
               Matrix        LM
(Intercept) 1.8551311 1.8551311
parent      0.1963383 0.1963383
sibling     0.2423249 0.2423249
peer        0.2229663 0.2229663
```

```r
#Compare residual variance
c(
  Matrix = matrix_model$residual_variance,
  LM = summary(lm_model)$sigma^2
)
```

```
   Matrix        LM
0.6846863 0.6846863
```

```r
#Compare fitted values
all.equal(
  as.vector(matrix_model$fitted_values),
  as.vector(fitted(lm_model))
)
```

```
[1] TRUE
```

```r
#Demonstrate prediction function
predictions <- predict_matrix_reg(matrix_model, X)

#Compare predictions with lm()
all.equal(
  predictions,
  as.vector(predict(lm_model))
)
```

```
[1] TRUE
```

The matrix-based regression function reproduced the results obtained from `lm()` to numerical tolerance. The estimated regression coefficients were 3.418 for the intercept, 0.471 for parent support, 0.23 for sibling support, and 0.524 for peer support. The corresponding standard errors were 1.855, 0.196, 0.242, and 0.223, respectively. The residual variance was 0.685. Both the fitted values and predictions agreed with the packaged regression results, confirming that the matrix-based fitting and prediction functions reproduce the results of `lm()`.

Core: Task 4

Write at least three unit tests that run, and visibly pass, in the rendered document: (a) your functions recover the packaged software's results; (b) your outputs have the correct dimensions; (c) your functions recover known parameters from data you simulate yourself. Reference Week 7's testing patterns here.

```r
#Test A: Agreement with lm()
stopifnot(
  isTRUE(all.equal(
    as.vector(matrix_model$coefficients),
    as.vector(coef(lm_model))
  )),
  isTRUE(all.equal(
```

```
    predictions,
    as.vector(predict(lm_model))
  ))
)

cat("Test A passed: Matrix estimates and predictions agree with lm().\n")
```

```
Test A passed: Matrix estimates and predictions agree with lm().
```

```
#Test B: Correct output dimensions
stopifnot(
  length(matrix_model$coefficients) == ncol(X) + 1,
  length(matrix_model$standard_errors) == ncol(X) + 1,
  length(matrix_model$fitted_values) == nrow(X),
  length(predictions) == nrow(X)
)

cat("Test B passed: Output dimensions are correct.\n")
```

```
Test B passed: Output dimensions are correct.
```

```
#Test C: Recovery of known parameters
set.seed(1234)

n <- 1000

#Simulate predictors
x1 <- rnorm(n)
x2 <- rnorm(n)

#Simulate outcome with known coefficients
y_sim <- 1 + 0.5*x1 - 0.3*x2 + rnorm(n, sd = 1)

#Fit matrix regression
sim_model <- fit_matrix_reg(
  y_sim,
  cbind(x1, x2)
)

#True population coefficients
true_beta <- c(1, 0.5, -0.3)
```

```r
#Compare estimated and true coefficients
data.frame(
  True = true_beta,
  Estimated = as.vector(sim_model$coefficients)
)
```

```
  True  Estimated
1  1.0  1.0299573
2  0.5  0.5177020
3 -0.3 -0.3368974
```

```r
#Check parameter recovery within tolerance
stopifnot(
  all(abs(as.vector(sim_model$coefficients) - true_beta) < 0.15)
)

cat("Test C passed: Known parameters recovered within tolerance.\n")
```

```
Test C passed: Known parameters recovered within tolerance.
```

All three unit tests passed. The matrix-based functions reproduced the coefficients and predictions obtained from `lm()`, returned outputs with the expected dimensions, and recovered the known population parameters from simulated data within the specified tolerance. Specifically, the estimated intercept was 1.03 compared with a true value of 1, the estimated coefficient for x1 was 0.518 compared with 0.5, and the estimated coefficient for x2 was -0.337 compared with -0.3. These results provide additional evidence that the functions perform as intended.

Stretch: Task 1

Full inference, by hand. The fitting function should also return the inferential quantities: t statistics, p-values, the model F test, and R2 — all computed from the matrix results, not pulled from packaged software — and the rendered document must verify each of them against the packaged output.

```r
#Extract packaged regression results
lm_summary <- summary(lm_model)

#Compare t statistics
t_comparison <- data.frame(
```

```
  Matrix = as.vector(matrix_model$t_statistics),
  LM = as.vector(lm_summary$coefficients[, "t value"])
)

rownames(t_comparison) <- names(coef(lm_model))

t_comparison
```

```
               Matrix        LM
(Intercept) 1.8425771 1.8425771
parent      2.3979183 2.3979183
sibling     0.9497146 0.9497146
peer        2.3487703 2.3487703
```

```
#Compare p-values
p_comparison <- data.frame(
  Matrix = as.vector(matrix_model$p_values),
  LM = as.vector(lm_summary$coefficients[, "Pr(>|t|)"])
)

rownames(p_comparison) <- names(coef(lm_model))

p_comparison
```

```
                Matrix         LM
(Intercept) 0.08399928 0.08399928
parent      0.02903933 0.02903933
sibling     0.35638329 0.35638329
peer        0.03201352 0.03201352
```

```
#Compare model F statistic
f_comparison <- data.frame(
  Matrix = matrix_model$F_statistic,
  LM = unname(lm_summary$fstatistic["value"])
)

f_comparison
```

```
    Matrix       LM
1 5.898418 5.898418
```

```r
#Compare R-squared
r2_comparison <- data.frame(
  Matrix = matrix_model$R_squared,
  LM = lm_summary$r.squared
)

r2_comparison
```

```
     Matrix        LM
1 0.5251557 0.5251557
```

```r
#Verify numerical agreement
stopifnot(
  isTRUE(all.equal(
    as.vector(matrix_model$t_statistics),
    as.vector(lm_summary$coefficients[, "t value"])
  )),
  isTRUE(all.equal(
    as.vector(matrix_model$p_values),
    as.vector(lm_summary$coefficients[, "Pr(>|t|)"])
  )),
  isTRUE(all.equal(
    matrix_model$F_statistic,
    unname(lm_summary$fstatistic["value"])
  )),
  isTRUE(all.equal(
    matrix_model$R_squared,
    lm_summary$r.squared
  ))
)

cat("All inference comparisons passed.\n")
```

```
All inference comparisons passed.
```

The matrix-based calculations reproduced the t statistics, p-values, model F statistic, and R^2 obtained from `lm()` to numerical tolerance. The overall model F statistic was 5.898, and the R^2 value was 0.525, indicating that approximately 52.5% of the variation in motivation was explained by parent, sibling, and peer support. The comparisons of coefficient-level t statistics and p-values also demonstrated numerical agreement with the packaged regression results. Together, these findings confirm that the function reproduces the inferential statistics calculated by `lm()` using matrix operations.

Stretch: Task 2

Loud failure. Given data containing missing values, or a predictor matrix that is rank- deficient, your functions must fail loudly with an informative error message rather than silently returning garbage — and your document must include tests proving both behaviors: one showing the missing-data refusal, one showing the rank-deficiency refusal, each demonstrating that the error occurs and says something useful.

```
#Test 1: Missing values
X_missing <- X
X_missing[1, 1] <- NA

missing_error <- tryCatch(
  fit_matrix_reg(y, X_missing),
  error = function(e) e
)

#Display error message
print(conditionMessage(missing_error))
```

```
[1] "Missing or non-finite values are not allowed."
```

```
#Verify an error occurred and the message is informative
stopifnot(
  inherits(missing_error, "error"),
  grepl("Missing", conditionMessage(missing_error))
)

cat("Test 1 passed: The function correctly rejected missing data and returned an informative
```

```
Test 1 passed: The function correctly rejected missing data and returned an informative erro
```

```
#Test 2: Rank-deficient predictor matrix
X_rank <- cbind(
  parent = motivation_data$parent,
  duplicate_parent = motivation_data$parent
)

rank_error <- tryCatch(
  fit_matrix_reg(y, X_rank),
  error = function(e) e
```

```
)

#Display error message
print(conditionMessage(rank_error))
```

```
[1] "Predictor matrix is rank-deficient."
```

```
#Verify an error occurred and the message is informative
stopifnot(
  inherits(rank_error, "error"),
  grepl("rank-deficient", conditionMessage(rank_error))
)

cat("Test 2 passed: The function correctly identified a rank-deficient predictor matrix and r
```

```
Test 2 passed: The function correctly identified a rank-deficient predictor matrix and retur
```

Both input-validation tests passed. When a missing value was introduced into the predictor matrix, the function rejected the input and returned an error indicating that missing or non-finite values were not allowed. Similarly, when two identical predictor columns were supplied, the function identified the rank-deficient matrix and returned an informative error. These tests demonstrate that the function appropriately rejects invalid inputs rather than producing potentially misleading regression estimates.